Practical 10

Part A:

Aim:

1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.

Code:

ThreadPoolRunner.java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner {

    public static void main(String[] args) {

        ExecutorService pool = Executors.newFixedThreadPool(3);

        try {
            for (int i = 1; i <= 10; i++) {
                final int taskId = i;

                pool.submit(() -> {
                    String threadName =
                            Thread.currentThread().getName();

                    System.out.println(
                            "Task " + taskId
                            + " executed by " + threadName
                    );
```

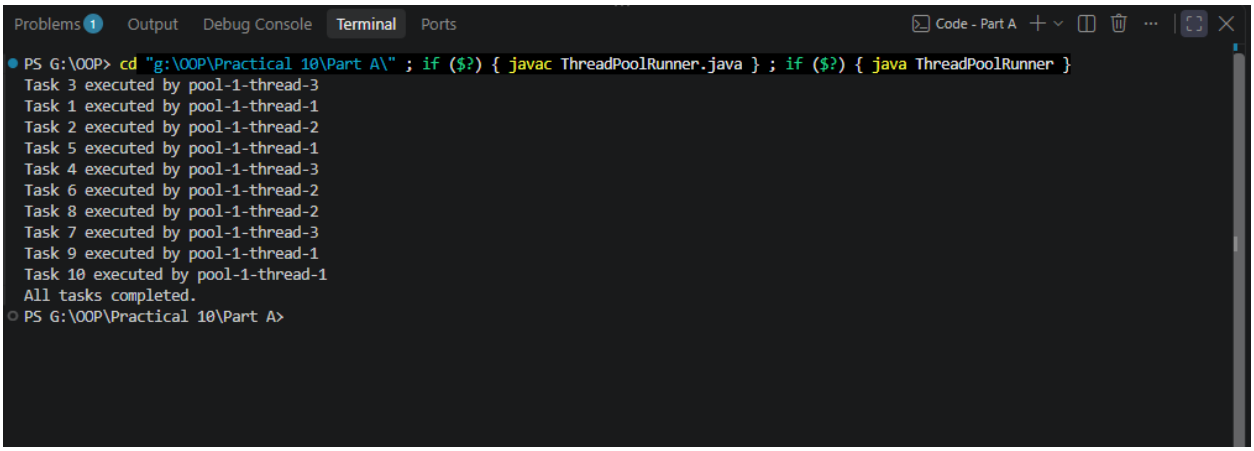
```
                    try {
                        Thread.sleep(500);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        System.out.println(
                                "Task " + taskId + "
interrupted."
                        );
                    }
                });
            }

        } finally {
            // Stop accepting new tasks
            pool.shutdown();

            try {
                // Wait for submitted tasks to finish
                if (!pool.awaitTermination(30,
TimeUnit.SECONDS)) {
                    pool.shutdownNow();
                }
            } catch (InterruptedException e) {
                pool.shutdownNow();
                Thread.currentThread().interrupt();
            }
        }

        System.out.println("All tasks completed.");
    }
}
```

Output:


Problems 1
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 10\Part A\" ; if ($?) { javac ThreadPoolRunner.java } ; if ($?) { java ThreadPoolRunner }
Task 3 executed by pool-1-thread-3
Task 1 executed by pool-1-thread-1
Task 2 executed by pool-1-thread-2
Task 5 executed by pool-1-thread-1
Task 4 executed by pool-1-thread-3
Task 6 executed by pool-1-thread-2
Task 8 executed by pool-1-thread-2
Task 7 executed by pool-1-thread-3
Task 9 executed by pool-1-thread-1
Task 10 executed by pool-1-thread-1
All tasks completed.
PS G:\OOP\Practical 10\Part A>

2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

Code:

ProducerConsumer.java

```
public class ProducerConsumer {

    static class SharedBuffer {

        private final int[] buffer = new int[3];
        private int count = 0;
        private int in = 0;
        private int out = 0;

        public synchronized void put(int item)
                throws InterruptedException {

            // Wait while the buffer is full
            while (count == buffer.length) {
                wait();
            }

            buffer[in] = item;
            in = (in + 1) % buffer.length;
            count++;

            System.out.println("Produced: " + item);

            // Notify waiting consumers or producers
            notifyAll();
        }

        public synchronized int take()
                throws InterruptedException {
```

```
        // Wait while the buffer is empty
        while (count == 0) {
            wait();
        }

        int item = buffer[out];
        out = (out + 1) % buffer.length;
        count--;

        System.out.println("Consumed: " + item);

        notifyAll();

        return item;
    }
}

public static void main(String[] args)
        throws InterruptedException {

    SharedBuffer buffer = new SharedBuffer();
    final int totalItems = 10;

    Thread producer = new Thread(() -> {
        try {
            for (int i = 1; i <= totalItems; i++) {
                buffer.put(i);
                Thread.sleep(100);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Producer");
```

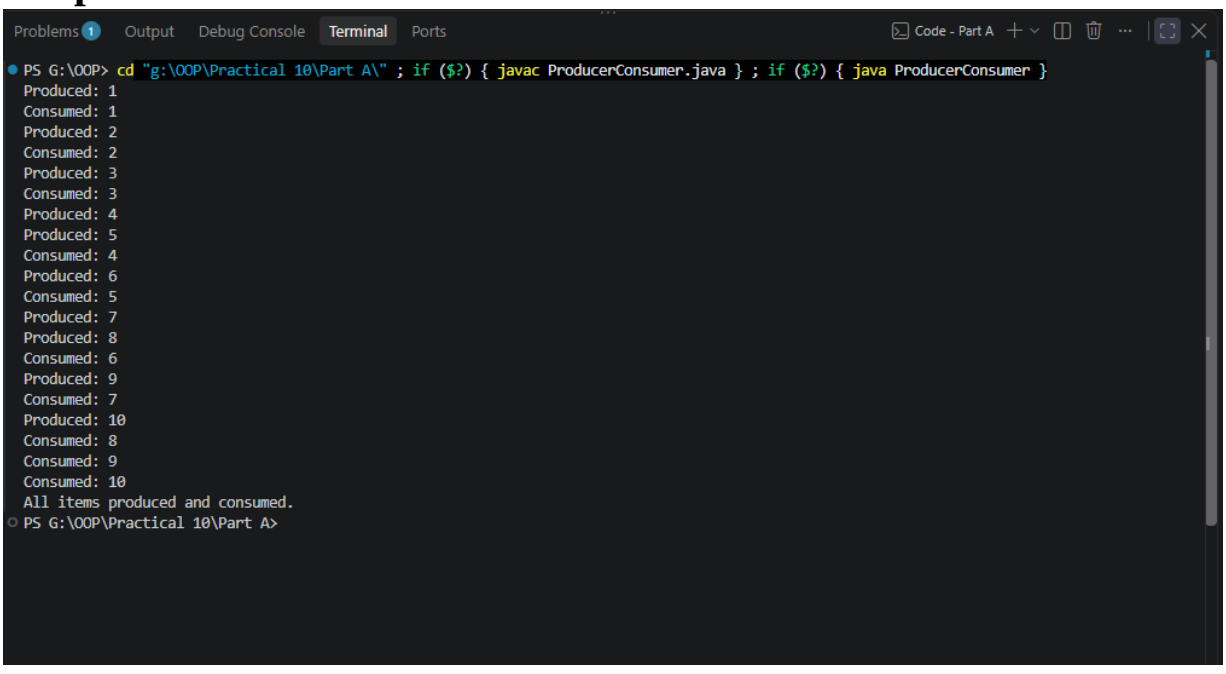
```
        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= totalItems; i++) {
                    buffer.take();
                    Thread.sleep(150);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Consumer");

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("All items produced and consumed.");
    }
}
```

Output:



3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.

Code:

DeadlockDemo.java

```
import java.util.concurrent.CountDownLatch;

public class DeadlockDemo {

    static class Account {
        final String name;

        Account(String name) {
            this.name = name;
        }
    }

    static void transferUnsafe(
            Account first,
            Account second,
            CountDownLatch firstLocksAcquired) {

        synchronized (first) {
            System.out.println(
                    Thread.currentThread().getName()
                    + " locked " + first.name
            );

            // Wait until both threads hold their first lock
            firstLocksAcquired.countDown();

            try {
                firstLocksAcquired.await();
            } catch (InterruptedException e) {
```

```
                Thread.currentThread().interrupt();
                return;
            }

            System.out.println(
                    Thread.currentThread().getName()
                    + " waiting for " + second.name
            );

            synchronized (second) {
                System.out.println("Transfer completed.");
            }
        }
    }

    // Always acquire locks in the same order
    static void transferSafe(
            Account first,
            Account second,
            Account lock1,
            Account lock2) {

        synchronized (lock1) {
            synchronized (lock2) {
                System.out.println(
                        Thread.currentThread().getName()
                        + " safely transferred between "
                        + first.name + " and " + second.name
                );
            }
        }
    }

    public static void main(String[] args)
            throws InterruptedException {
```

```
        Account a = new Account("Account A");
        Account b = new Account("Account B");

        System.out.println("=== Demonstrating Deadlock ===");

        CountDownLatch latch = new CountDownLatch(2);

        Thread t1 = new Thread(
                () -> transferUnsafe(a, b, latch),
                "Thread-1"
        );

        Thread t2 = new Thread(
                () -> transferUnsafe(b, a, latch),
                "Thread-2"
        );

        // Daemon threads prevent the demonstration from
        // keeping the JVM alive after main() finishes.
        t1.setDaemon(true);
        t2.setDaemon(true);

        t1.start();
        t2.start();

        t1.join(1000);
        t2.join(1000);

        if (t1.isAlive() && t2.isAlive()) {
            System.out.println(
                    "Deadlock detected: both threads are
blocked."
            );
        } else {
```

```
            System.out.println(
                    "The deadlock was not confirmed in this
run."
            );
        }

        System.out.println("\n=== Preventing Deadlock ===");

        Account safeA = new Account("Safe A");
        Account safeB = new Account("Safe B");

        // Both threads always lock safeA first, then safeB.
        Thread safeT1 = new Thread(
                () -> transferSafe(safeA, safeB, safeA, safeB),
                "Safe-Thread-1"
        );

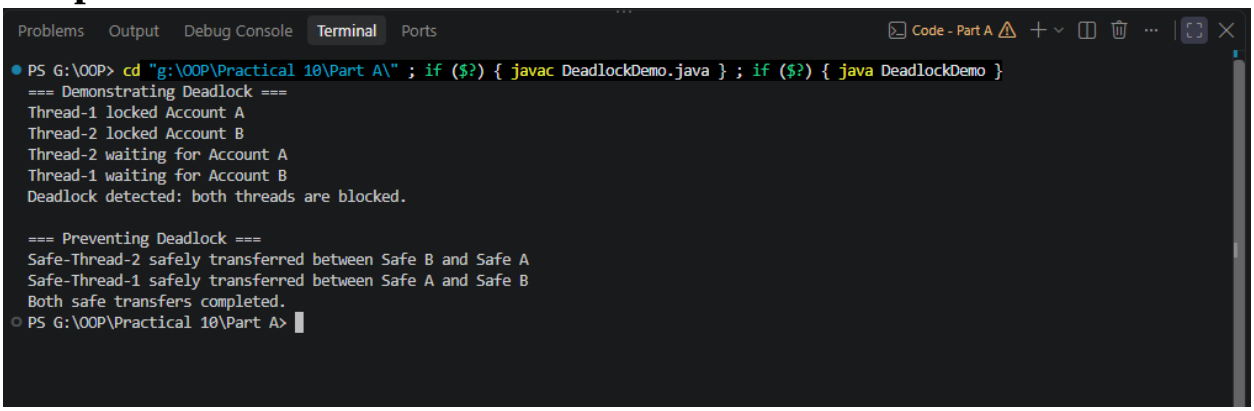
        Thread safeT2 = new Thread(
                () -> transferSafe(safeB, safeA, safeA, safeB),
                "Safe-Thread-2"
        );

        safeT1.start();
        safeT2.start();

        safeT1.join();
        safeT2.join();

        System.out.println("Both safe transfers completed.");
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. Why is a thread pool better than creating a new thread per task?

Ans.

 Reduced overhead: Reusing threads avoids repeatedly creating and destroying them.

 Resource management: Limits the number of concurrently active worker threads.

 Improved scalability: Can handle many tasks without creating an excessive number of threads.

 Task management: ExecutorService provides methods to submit tasks, shut down the pool, and wait for completion.

2. How do wait() and notify() coordinate two threads?

Ans.

wait() and notify() are methods of Java's Object class that support communication between threads using an object's monitor.

- wait() releases the object's monitor and makes the current thread wait until it is notified or otherwise awakened.
- notify() wakes one thread waiting on that object's monitor.
- notifyAll() wakes all threads waiting on that object's monitor.

These methods must be called while holding the same object's monitor, usually inside a synchronized block or method.

3. What causes a deadlock, and how does consistent lock ordering prevent it?

Ans.

A deadlock occurs when two or more threads wait indefinitely for resources or locks held by one another.

Example:

- Thread 1 locks Account A and waits for Account B.
- Thread 2 locks Account B and waits for Account A.
- Neither thread can proceed because each is waiting for the other to release its lock.

Four conditions are commonly associated with deadlock:

1. Mutual exclusion: A resource can be held by only one thread at a time.
2. Hold and wait: A thread holds one resource while waiting for another.
3. No preemption: A resource cannot simply be taken away from its holder.
4. Circular wait: A cycle exists in which each thread waits for a resource held by the next thread.

How consistent lock ordering prevents deadlock

Require every thread to acquire locks in the same predefined order.